Guía de ejercicios 2 — Patrones estructurales

Programación II — TSSI · Unidad 4

Indicaciones

Los patrones estructurales resuelven cómo combinar clases y objetos en estructuras mayores sin modificar lo que ya existe. Esta guía tiene un ejercicio por patrón.

- Resolver cada ejercicio en una aplicación de consola C# (`dotnet new console`).
- Los enunciados no dicen qué clases crear: decidir las clases, interfaces y relaciones es parte del ejercicio. Antes de programar, dibujar el diagrama de clases.
- Programar contra interfaces o clases abstractas; el `Main` solo arma objetos y muestra resultados.
- Una clase por archivo, con nombres en inglés como en el material de lectura (pág. 78-81).
- Comparar lo que muestra tu programa con la salida esperada y responder las preguntas de "Para pensar".

Ejercicios principales

Ejercicio 1 — Adapter: termómetro importado

El sistema de monitoreo de un laboratorio muestra en un listado la temperatura de todos sus sensores. Todos los sensores propios se usan a través de la misma interfaz, que devuelve grados Celsius; un sensor propio marca hoy 25 °C.

Se compró un termómetro importado. Su librería expone un método con otro nombre que devuelve grados Fahrenheit (hoy, 98.6) y no se puede modificar. El listado tiene que incluir el termómetro nuevo sin casos especiales ni cambios en su código. Conversión: (F − 32) × 5 / 9.

Para verificar: mostrar el listado con un sensor propio y el termómetro importado.

```
Sensor local: 25.0 °C
Sensor importado: 37.0 °C
```

Para pensar: ¿por qué conviene que la pieza nueva componga al termómetro importado en lugar de heredar de él?

Ejercicio 2 — Decorator: café con agregados

Una cafetería vende café ($1500) y permite sumarle agregados: leche (+$300), crema (+ $400) y chocolate (+$500), en cualquier combinación y hasta repetidos (doble chocolate). El ticket muestra la descripción completa y el precio final.

La carta de agregados cambia seguido. Sumar un agregado nuevo no debe obligar a crear una clase por cada combinación posible ni a modificar las clases existentes.

Para verificar: armar un café con leche y chocolate y mostrar su ticket.

```
Café, leche, chocolate: $2300
```

Para pensar: ¿cuántas clases harían falta para cubrir todas las combinaciones usando solo herencia? ¿Y con tu solución?

Ejercicios complementarios

Ejercicio 3 — Composite: menú de una aplicación

Una aplicación de escritorio tiene un menú con opciones simples y submenús. Un submenú puede contener opciones y otros submenús, sin límite de profundidad.

Hay que mostrar el menú completo, con una sangría de dos espacios por nivel. El código que muestra el menú no debe preguntar si cada elemento es una opción o un submenú: alcanza con pedirle a la raíz que se muestre.

Para verificar: armar este menú y mostrarlo desde la raíz.

```
Archivo
  Nuevo
  Abrir
  Recientes
    informe.txt
    notas.txt
  Salir
```

Para pensar: si las opciones simples tuvieran que ofrecer “agregar hijo”, ¿qué deberían hacer? Relacionarlo con ISP, como señala el material.

Ejercicio 4 — Facade: home theater

Un home theater tiene proyector, equipo de sonido y luces, y cada aparato tiene sus propias operaciones: encender, elegir entrada, ajustar el volumen, atenuar. Para ver una película siempre hay que hacer la misma secuencia: luces al 10 %, proyector encendido en HDMI y sonido encendido con volumen 15. Al terminar, hay que deshacerla.

La familia que lo usa no quiere conocer los aparatos: solo quiere “ver película” y “terminar película”. Los aparatos tienen que seguir pudiendo usarse por separado.

Para verificar: iniciar una película desde el código cliente con una sola llamada.

```
Luces al 10%
Proyector encendido en HDMI
Sonido encendido, volumen 15
-- Disfrutá la película --
```

Para pensar: ¿qué pasa con tu solución si mañana se agrega un aparato nuevo, por ejemplo una consola? ¿Cuándo empieza a tener demasiadas responsabilidades?